A PROJECT REPORT ON

"StrayCare: A Community-Driven Platform for Animal Rescue and Adoption"

SUBMITTED BY

BIB04 Sanath Bhat

BIB21 Karttikey Chavali

BIB23 Hariom Khaladkar

BIB26 Rajat Kulkarni

GUIDED BY

Dr. Bhavana Kanawade

DEPARTMENT OF INFORMATION TECHNOLOGY

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY, PUNE - 411057

SAVITRIBAI PHULE PUNE UNIVERSITY

2025 – 2026

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY

HINJEWADI, PUNE-57

DEPARTMENT OF INFORMATION TECHNOLOGY

Certificate

This is to certify that the project report entitled **"StrayCare: A Community-Driven Platform for Animal Rescue and Adoption"** which is being submitted by:

- **BIB04 Sanath Bhat**
- **BIB21 Karttikey Chavali**
- **BIB23 Hariom Khaladkar**
- **BIB26 Rajat Kulkarni**

have completed Phase II of the Project entitled **"StrayCare: A Community-Driven Platform for Animal Rescue and Adoption"**, under my guidance in partial fulfillment of the requirement for the award of the Bachelor of Engineering in Information Technology of International Institute of Information Technology, Hinjawadi, by Savitribai Phule Pune University for the academic year 2025 – 2026.

Dr. Bhavana Kanawade
Guide

Internal Examiner: _____

Dr. Jyoti Surve
Head of the Department

External Examiner: _____

Dr. Vaishali Patil

Principal

Date: _____

Acknowledgement

With immense pleasure, we are presenting this Phase II project report on "StrayCare: A Community-Driven Platform for Animal Rescue and Adoption" as a part of the curriculum of B.E. Information Technology at International Institute of Information Technology, Hinjewadi, Pune.

It gives us the privilege to complete this report work under the valuable mentorship of **Dr. Bhavana Kanawade**. Her guidance, co-operation, and encouragement have made a significant headway in the project. We are also extremely grateful to **Dr. Vaishali Patil**, Principal, and **Dr. Jyoti Surve**, Head of Department of Information Technology, for providing all facilities and help for smooth progress of work. We would also like to thank all the Staff Members of the Information Technology Department, Management, friends, and our family members, who have directly or indirectly guided and helped us for the preparation of this report and gave us unending support right from the stage the idea was conceived.

Phase II represents a major evolution of the StrayCare platform, introducing advanced capabilities including AI-powered chatbot integration, Razorpay payment gateway for NGO donations, Google OAuth 2.0 single sign-on, comprehensive analytics dashboards for all user roles, an interactive Zone Red Hotspot Map powered by K-Means clustering, a Smart NGO Dispatch system using the Haversine distance formula, and a Donor Transparency Dashboard. We are grateful to all who supported this extended development effort.

BIB04 Sanath Bhat

BIB21 Karttikey Chavali

BIB23 Hariom Khaladkar

BIB26 Rajat Kulkarni

Abstract

StrayCare is an innovative, full-stack web platform developed to connect compassionate citizens with verified animal welfare organizations, creating a seamless and transparent channel for reporting, managing, and resolving stray animal rescue cases. In Phase I, the platform established a robust foundation: a citizen case reporting module with GPS tagging, an NGO case management dashboard, a pet adoption gallery, an admin verification panel, and a feedback system — all secured with JWT-based role-based access control.

Phase II marks a significant expansion of the platform's intelligence, financial infrastructure, and analytical capabilities. The platform now integrates a Groq-powered AI chatbot (Llama 3.3 70B model) for multi-turn animal first-aid and rescue conversations, replacing the earlier rule-based system. A full payment gateway powered by Razorpay has been implemented, enabling citizens to make NGO-specific donations via UPI, cards, and net banking, with complete order creation, webhook verification, and idempotent status tracking. Google OAuth 2.0 has been integrated to allow frictionless single sign-on for citizens.

Advanced analytics dashboards have been introduced for all three user roles. The NGO Analytics Dashboard visualizes case trends (month-wise and year-wise), species distribution, adoption rates, and donation history using interactive charts. The Admin Analytics Dashboard provides a platform-wide bird's-eye view of case volumes, donation totals, and adoption statistics. The Donor Transparency Dashboard is a public-facing portal that builds public trust by displaying aggregated impact metrics, month-wise donation charts, and total animals rescued.

Two new ML-powered features have been deployed: the Zone Red Hotspot Map, which uses a custom K-Means clustering algorithm to identify geographical danger zones from reported case coordinates and renders them as color-coded risk overlays on an interactive Leaflet.js map; and the Smart NGO Dispatch system, which uses the Haversine great-circle distance formula combined with a weighted scoring matrix (proximity, active caseload, and base score) to present administrators with a ranked list of NGOs for any given emergency. Citizens can now also list their own pets for adoption, subject to NGO verification and approval. NGOs can respond directly to citizen feedback, completing the accountability loop.

The platform's AI animal species recognition (TensorFlow.js + MobileNet V2) and Rule-Based NLP severity triaging from Phase I continue to operate, now augmented by the Groq LLM backend for richer conversational support. Built on React.js (frontend), FastAPI/Python (backend), SQLite with SQLAlchemy ORM, and deployed with Uvicorn, StrayCare is a comprehensive, production-ready civic technology platform that demonstrates how thoughtful software engineering can transform community compassion into real-world animal welfare impact.

Chapter 1: Introduction to Project Topic

1.1 Overview

1.1.1 Project Overview

Across India's cities and towns, countless stray animals suffer from injuries, illness, and neglect, often in plain sight of residents who feel powerless to help. While social media and local chat groups have become informal channels for reporting such incidents, these methods are fragmented, unreliable, and lack a structured response mechanism. Compassionate citizens who witness an animal in distress often feel helpless — unsure of whom to contact or how to provide effective aid. This communication gap between the public and animal welfare organizations leads to delayed rescues, wasted resources, and preventable suffering.

StrayCare was envisioned in Phase I as a transformative digital solution to bridge this gap, democratizing the process of animal rescue. Phase II deepens this vision by adding a comprehensive financial layer (Razorpay donations), AI conversational support (Groq LLM chatbot), advanced analytics for all stakeholders, geospatial intelligence (K-Means hotspot mapping, Haversine dispatch), and citizen pet listing capabilities. The platform now represents a complete, production-grade civic technology ecosystem for animal welfare.

1.2 Brief Description

StrayCare is a comprehensive animal rescue, adoption, and fundraising portal developed using modern web technologies. It is built on a role-based access control system for three primary user types: **Citizens**, **NGOs**, and **Administrators**. Citizens can create geo-tagged case reports with photos, track the status of their reports in real time, browse adoptable pets with AI-powered lifestyle matching, list their own pets for adoption, donate to NGOs via integrated payment gateways, and interact with an AI-powered first-aid chatbot.

NGOs receive notifications for cases in their operational area, manage their entire rescue caseload through a dedicated dashboard, post progress updates with photos, list rescued animals for adoption, manage adoption requests, respond to citizen feedback, view their own analytics dashboard, and track incoming donations. Administrators oversee the entire ecosystem: verifying new NGOs, managing platform users, viewing all feedback, dispatching NGOs to emergencies using the Smart Dispatch tool, and monitoring platform-wide analytics.

1.3 Problem Definition

This project directly addresses systemic inefficiencies in the current, informal stray animal rescue process. The key problems are:

- **Fragmented and Unreliable Reporting:** Citizens rely on scattered social media posts or unverified contact numbers, with no guarantee that their report will reach the right people in time.

- **Lack of Actionable Information for NGOs:** Rescuers are often inundated with duplicate reports, calls from outside their service area, or messages lacking critical details like a precise location or photo, leading to wasted time and resources.
- **No Feedback Loop for Citizens:** Individuals who report a case rarely receive updates on the animal's condition, leading to a sense of futility and discouraging future reporting.
- **Absence of a Transparent Donation Mechanism:** Animal welfare NGOs lack a credible, integrated platform to receive and acknowledge public donations, while donors have no visibility into the impact of their contributions.
- **Inefficient NGO Dispatching:** When multiple NGOs operate in an area, there is no intelligent system to route emergencies to the most suitable organization based on proximity and current caseload.
- **No Consolidated Analytics:** NGOs and administrators lack data-driven insights into case trends, geographic hotspots, species distributions, and fundraising performance, hindering effective resource planning.

1.4 Applying Software Engineering Approach

StrayCare is developed using a structured Agile software engineering methodology:

- **Requirements Engineering:** A thorough analysis of all three key actors' needs produced clear functional and non-functional requirements spanning both Phase I and Phase II features.
- **API-First Architectural Design:** The three-tier architecture (React frontend, FastAPI backend, SQLite data layer) ensures modularity and extensibility. All Phase II features were implemented as new FastAPI router modules, keeping existing functionality intact.
- **Database Migration Safety:** Phase II added multiple new columns to existing tables (e.g., severity_score, upi_id, video_url) using safe ALTER TABLE migrations executed at application startup, ensuring zero downtime for existing data.
- **Technology Stack Evolution:** Phase II introduced Razorpay SDK, google-auth library, Groq SDK, and Recharts (React charting library) to the stack without disrupting existing dependencies.
- **Quality Assurance:** All Phase II features underwent unit testing (algorithm accuracy), API endpoint testing via FastAPI Swagger UI and Postman, and end-to-end manual testing across multiple browsers and device types.

Chapter 2: Literature Survey

During the requirement gathering phase for StrayCare Phase II, a comprehensive review of existing digital animal welfare solutions, civic technology platforms, and AI-powered emergency response systems was conducted. This review analyzed research papers and deployed applications to understand current advancements, persistent challenges, and technological gaps.

2.1 Incident Reporting and Alert Systems

Current methods for reporting animals in distress are often disconnected from actual, verified rescue operations. While a significant body of research focuses on mobile reporting apps, these systems often function as "alert" mechanisms without a clear case management backend. Advanced systems are now integrating location-based technology and AI for real-time assistance. The work by Rajwani et al. on using AI for real-time reporting, Sengan et al. on deep learning for roadway animal detection, and Chen, Liu & Liao on tracking stray animals shows a clear evolution toward intelligent, multi-modal platforms. StrayCare's Phase II Smart NGO Dispatch system directly responds to this need by intelligently routing emergencies to the optimal responder.

2.2 Case Management and Citizen Tracking

A major gap identified in the literature is the lack of transparency for the citizen after a report is made. Many organizations manage cases using internal-only tools, leaving the reporter uninformed. The work by Wu et al. emphasized the need for a unified rescue system that formally connects the citizen's report to the NGO's workflow. StrayCare solves this through real-time case status tracking, progress update timelines, and after-resolution feedback — all accessible directly from the citizen's dashboard.

2.3 Adoption and Rehoming Processes

Many platforms focus on building better adoption storefronts, but treat adoption as completely separate from rescue. Advanced systems by Jha et al. (AI-based adoption matching), Lu (machine learning for adoption status prediction), and Torres et al. (content-based filtering) demonstrate the move toward intelligent matching. StrayCare Phase II goes further by integrating a citizen pet listing portal — enabling not just NGO-driven but also community-driven pet rehoming, subject to NGO verification, creating a comprehensive adoption ecosystem.

2.4 Donation Transparency and Accountability

The literature reveals a critical gap in digital civic platforms: the absence of transparent, verifiable donation mechanisms. Most NGO platforms rely on offline fund collection or generic donation links with no impact reporting. StrayCare Phase II addresses this through an end-to-end Razorpay payment integration with NGO-specific order creation, webhook-verified

status updates, UPI fallback support, and a public Donor Transparency Dashboard that aggregates month-wise and year-wise donation statistics alongside case resolution and adoption impact metrics.

2.5 AI-Powered Conversational Interfaces for Emergency Response

Studies on Large Language Model (LLM) integration in emergency response platforms show that domain-specific AI assistants significantly improve citizen engagement and first-response outcomes. The work on Llama-series models demonstrates that open-weight LLMs can match proprietary model performance for focused tasks at a fraction of the cost. StrayCare Phase II integrates the Groq API (Llama 3.3 70B) specifically for multi-turn animal rescue and first-aid conversations, replacing the earlier rule-based chatbot with one that handles nuanced, context-aware queries while maintaining conversation history.

2.6 Comparative Analysis of Existing Systems

System/Paper	Technology	Key Feature	Gap Addressed in StrayCare
Pet Adoption & Rescue Center (Prajapati et al.)	Node.js, MongoDB, Google Maps API	Proximity search, AI image recognition	Unified rescue-to-adoption workflow, analytics
CareForPaws (Blancaflor et al.)	React Native, GPS, SUS-tested	Location-based matching, SUS score: 85	Donation gateway, groq AI chatbot, hotspot map
FurRescue (Jean & Wahid)	Mobile app, AI breed detection	Geo-fencing, push notifications	NGO dispatch ranking, K-Means clustering
SRMS (Subramaniam et al.)	Web app, DB-driven	Structured rescue management	Donor dashboard, NGO analytics, Google OAuth
PetHub (prior work)	PHP, MySQL	Streamlined adoption & donations	Real-time case updates, LLM chatbot, feedback responses

Table 2.1: Comparative Analysis of Existing Systems and StrayCare Phase II Contributions

The comparative analysis confirms that while individual platforms have solved specific aspects of the problem, no existing system integrates the complete set of features: structured reporting, NGO case management, AI-powered severity triaging, intelligent dispatch, geospatial hotspot visualization, transparent donation management, AI conversational support, and comprehensive analytics. StrayCare Phase II bridges all these gaps within a single, cohesive platform.

Chapter 3: Software Requirements Specification

3.1 Introduction

3.1.1 Purpose

The primary purpose of StrayCare Phase II is to transform the Phase I prototype into a fully production-capable civic technology platform. This phase adds financial infrastructure (Razorpay donations), enterprise-grade authentication (Google OAuth 2.0), AI/ML-powered intelligence (Groq LLM chatbot, advanced dispatch, hotspot mapping), and rich analytics for all stakeholders. The system continues to bridge the gap between compassionate citizens and verified NGOs while now also providing transparent resource allocation, donor accountability, and data-driven operational insights.

3.1.2 Intended Audience and Reading Suggestion

This document is intended for software developers, UI/UX designers, quality assurance testers, project managers, academic reviewers, animal welfare organizations, and potential government or municipal partners. Readers should begin with the project scope and system features before exploring technical constraints, security, and evaluation criteria.

3.1.3 Project Scope — Phase II Additions

Building upon the Phase I scope (case reporting, NGO dashboard, adoption gallery, admin panel, feedback system), Phase II adds the following modules:

- **Razorpay Donation Gateway:** End-to-end payment processing with order creation, Razorpay checkout integration, webhook-based status verification, and UPI deep-link fallback.
- **Google OAuth 2.0:** Single sign-on authentication for citizens using their Google accounts, with automatic account creation for new users.
- **Groq AI Chatbot:** Multi-turn conversational AI using the Llama 3.3 70B model via the Groq API, replacing the rule-based chatbot for richer, context-aware animal rescue guidance.
- **NGO Analytics Dashboard:** Visual charts showing case trends (month-wise, year-wise), species distribution, monthly adoptions, and monthly donations.
- **Admin Analytics Dashboard:** Platform-wide analytics showing total cases by status, total donations, and adoption statistics with interactive charts.
- **Donor Transparency Dashboard:** Public-facing portal showing aggregate donation impact, month-wise donation charts, cases supported, and adoptions facilitated.
- **Zone Red Hotspot Map:** Interactive Leaflet.js map with color-coded risk overlays computed from K-Means clustering of reported case GPS coordinates.
- **Smart NGO Dispatch:** Admin tool that ranks NGOs for a selected case using Haversine distance, active caseload, and base score to produce a weighted recommendation list.
- **Citizen Pet Listing:** Citizens can submit their own pets for adoption, subject to NGO verification and approval, broadening the adoption marketplace.

- **NGO Feedback Response:** NGOs can publicly respond to citizen reviews, completing the accountability and trust-building loop.
- **Pet Video Upload:** NGOs can attach short video clips to pet adoption listings, providing richer adoption profiles.
- **Donor KYC Verification:** A two-step OTP-based email and phone verification flow before high-value donations.

3.1.4 Design and Implementation Constraints

The Phase II system continues to use React (frontend) and FastAPI/Python (backend). All new Phase II database columns were added using safe SQLite ALTER TABLE migrations to preserve existing data. Razorpay API keys are loaded via environment variables for security. The Groq API key is stored server-side and never exposed to the client. The UPI payment deep-link fallback works on Android/iOS devices with a UPI app installed. All chart components use Recharts, a React-native charting library that requires no additional server-side dependencies. The Leaflet.js map runs entirely in the browser with map tiles fetched from OpenStreetMap.

3.1.5 Assumptions and Dependencies

The system assumes that users have internet access and a modern browser (Chrome, Firefox, Edge, Safari). Google OAuth requires a valid GOOGLE_CLIENT_ID environment variable. Razorpay integration requires RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, and RAZORPAY_WEBHOOK_SECRET. The Groq chatbot requires a valid GROQ_API_KEY. For production deployment, HTTPS is mandatory for Google OAuth callbacks and Razorpay webhooks.

3.2 System Features — Phase II

3.2.1 Razorpay Donation Payment System

The donation module enables secure, verifiable monetary contributions to specific NGOs:

- Citizens browse the donation page, which lists all verified NGOs with their 30-day donation totals and case statistics via GET /donations/ngos.
- After selecting an NGO and amount, the frontend calls POST /donations/create-order, which creates a Razorpay order server-side and returns the order ID to the client.
- The Razorpay Checkout SDK is loaded client-side to handle card, UPI, net banking, and wallet payments.
- Upon payment completion, Razorpay sends a webhook to POST /donations/webhook; the backend verifies the HMAC-SHA256 signature and updates the donation status to 'Completed' idempotently.
- If no Razorpay keys are configured, the frontend displays a UPI deep-link fallback using the NGO's registered UPI ID, enabling offline UPI payments.

3.2.2 Google OAuth 2.0 Authentication

Citizens can sign in using their Google account via the POST /auth/google endpoint. The backend verifies the Google ID token using the google-auth library, extracts the user's email

and name, creates a new account if one does not exist, and returns a JWT access token. This eliminates the need for citizens to remember a separate password while maintaining full compatibility with the existing JWT-based session system.

3.2.3 Groq AI Chatbot (LLM)

The chatbot at POST /chatbot/query now routes requests to the Groq API using the Llama 3.3 70B model (llama-3.3-70b-versatile). The complete conversation history is passed to the model on every request, enabling rich multi-turn conversations. The system prompt configures the model as a specialized StrayCare Animal Rescue Assistant, directing it to provide expert first-aid guidance for injured and distressed animals. On API failure, the system gracefully falls back to the local rule-based engine in chatbot_logic.py, ensuring zero downtime for users.

3.2.4 Analytics Dashboards

Three new analytics interfaces have been added using Recharts:

- **NGO Analytics Dashboard (/ngo/dashboard/analytics):** Displays line charts for monthly case trends, bar charts for species distribution, adoption trends, and monthly donation amounts — all powered by dedicated FastAPI endpoints consuming NGO-scoped database queries.
- **Admin Analytics Dashboard (/admin/dashboard/analytics):** Provides platform-wide totals for cases by status (Pending, Accepted, Resolved), total donation amounts, and adoption counts with interactive pie and bar charts.
- **Donor Transparency Dashboard (/donor-dashboard):** A public page (no authentication required) showing month-wise donation charts, total amount raised, cases supported, and adoption statistics to build donor confidence.

3.2.5 Zone Red Hotspot Map

An interactive Leaflet.js map visualizes animal rescue emergency hotspots. The backend endpoint GET /admin/hotspots fetches all reported cases with GPS coordinates, runs the K-Means clustering algorithm (k=5 by default), and returns colored cluster centroids with ring radius values. The frontend renders these as color-coded semi-transparent circles (red = Critical ≥10 cases, orange = High ≥5 cases, yellow = Moderate ≥2, green = Low). Clicking a cluster displays its case count and risk level. This tool enables NGOs and administrators to proactively allocate resources and plan shelter placement.

3.2.6 Smart NGO Dispatch

The Smart Dispatch page (GET /admin/smart-dispatch/{case_id}) allows administrators to intelligently route a specific emergency case to the most suitable NGO. The Haversine formula computes the great-circle distance between the case coordinates and each registered NGO's location. A weighted scoring matrix then ranks NGOs: 50 points for proximity (closer = higher), 30 points for low active caseload (fewer current cases = higher), and a 20-point base score for all verified NGOs. The ranked list is displayed with distance, active case count, and total dispatch score — enabling one-click case assignment.

3.3 External Interface Requirements

3.3.1 User Interface

StrayCare maintains its mobile-first, glassmorphism-inspired design throughout Phase II. New pages follow the same design system: the Analytics dashboards use a consistent card-and-chart layout with color-coded data series; the Hotspot Map fills the viewport with a sidebar statistics panel; the Smart Dispatch page uses a ranking table with a sticky action column; the Donor Dashboard is public-facing with high-contrast impact statistics. All new forms (Pet Listing, Feedback Response) are modals with inline validation and loading states.

3.3.2 Software Interface

Phase II introduces the following external service integrations:

- **Razorpay API:** Creates orders with razorpay.Client.order.create(), verifies webhooks using HMAC-SHA256 with the Razorpay webhook secret.
- **Google Auth Library:** google.oauth2.id_token.verify_oauth2_token() validates Google ID tokens server-side against the GOOGLE_CLIENT_ID.
- **Groq API:** groq.Groq().chat.completions.create() sends conversation history to the Llama 3.3 70B model with a system-configured persona.
- **Leaflet.js:** Browser-native map rendering with OpenStreetMap tiles; no server-side mapping dependencies required.
- **Recharts:** All charts are pure React components using SVG rendering, requiring no additional backend services.

3.3.3 Communication Interface

All client-server communication remains HTTPS-secured RESTful JSON APIs. New Phase II webhook endpoints (POST /donations/webhook) verify request authenticity using HMAC-SHA256 signature validation before processing payment status updates. The Groq API calls are made server-to-server (backend to Groq), ensuring the API key is never exposed to browser clients.

3.4 Non-Functional Requirements

3.4.1 Performance Requirements

Phase II maintains all Phase I performance targets: API response time under 500ms for standard data retrieval, FCP under 2 seconds on 3G. The Groq AI chatbot endpoint has a target response time of under 3 seconds for typical queries. The K-Means clustering algorithm processes up to 500 case coordinates in under 100ms using pure Python with no dependency on external ML libraries. Analytics dashboard pages use parallel API calls (Promise.all) to minimize total page load time.

3.4.2 Security Requirements

Phase II introduces additional security measures:

- **Webhook Signature Verification:** All Razorpay webhook requests are verified using HMAC-SHA256 with the RAZORPAY_WEBHOOK_SECRET before processing, preventing fraudulent payment status updates.

- **Environment Variable Isolation:** All API keys (Razorpay, Groq, Google OAuth) are stored exclusively in server-side environment variables and never included in client-side code bundles.
- **Idempotent Donation Processing:** The webhook handler checks existing donation status before updating, preventing double-counting from duplicate webhook deliveries.
- **OAuth Token Validation:** Google ID tokens are validated server-side using the official Google Auth library, not just decoded, ensuring token freshness and authenticity.

3.4.3 Software Quality Attributes

All Phase I quality attributes (Maintainability, Scalability, Reusability, Portability) are preserved. Phase II adds:

- **Resilience:** The Groq chatbot falls back to the local rule-based engine on API failure, and the donation system degrades gracefully to UPI deep-links if Razorpay keys are absent.
- **Observability:** The startup event logs all database migration outcomes and seeding status, enabling rapid diagnosis of deployment issues.

Chapter 4: System Design

4.1 System Architecture — Phase II

The Phase II architecture maintains the three-tier client-server structure of Phase I while introducing new external service integrations at the application logic layer. The system now consists of five logical layers:

- **1. Presentation Layer (React SPA):** Extended with new pages — AdminAnalyticsDashboard, NGOAnalyticsDashboard, DonorDashboard, HotspotMap, SmartDispatch, DonatePage, DonorVerification, UserPetListingForm — all using CSS Modules and Recharts for consistent, responsive UI.
- **2. Client-Side Intelligence Layer:** TensorFlow.js + MobileNet V2 (animal species recognition) and Leaflet.js (hotspot map rendering) continue to run in-browser, protecting user privacy and reducing server load.
- **3. API Gateway (FastAPI):** All business logic, algorithm execution, and third-party API orchestration. New Phase II router groups: /donations (Razorpay), /auth/google (OAuth), /chatbot (Groq LLM), /admin/hotspots (K-Means), /admin/smart-dispatch (Haversine), /ngo/dashboard and /admin/dashboard (analytics), /donor/dashboard (public stats), /donor/verify (KYC).
- **4. External Services Layer:** Razorpay Payment Gateway (order API + webhook), Groq AI API (LLM inference), Google Auth servers (OAuth 2.0 token verification).
- **5. Data & Storage Layer:** SQLite database (straycare.db) extended with new columns and tables: donations, user_pet_listings, feedback enhancements (category, ngo_response), NGO UPI ID; uploads/ directory for case photos, pet images, verification documents, and pet videos.

4.2 Database Schema — Phase II Extensions

Table	New Column(s) Added in Phase II	Purpose
cases	severity_score (INT), severity_label (TEXT), pet_name (TEXT), is_adoptable (INT), adoption_story (TEXT), temperament (TEXT)	NLP severity triaging, adoptability workflow
pets	ngo_id (INT), video_url (TEXT)	NGO ownership tracking, video adoption profiles
ngos	upi_id (TEXT)	Direct UPI payment fallback for donations
feedback	category (TEXT), ngo_response (TEXT)	Feedback categorization, NGO reply capability
donations	id, amount, currency, status, order_id, razorpay_payment_id, ngo_id, user_id, created_at	Full donation lifecycle tracking (NEW TABLE)
user_pet_listings	id, name, species, age, location, description, image_url, status, user_id, created_at	Citizen pet rehoming portal (NEW TABLE)

Table 4.1: Phase II Database Schema Extensions

4.3 UML Diagrams — Phase II Additions

4.3.1 Module 4: Donation & Payment Workflow

Use Case Diagram: Defines actors: 'Citizen/Donor' (browses NGOs, selects amount, initiates payment, views donation history) and 'Admin' (views all donations, monitors payment totals). An 'NGO' actor can view their own incoming donations via the NGO Analytics Dashboard.

Activity Diagram: Shows the complete donation lifecycle: Citizen selects NGO → enters amount → frontend calls POST /donations/create-order → receives Razorpay order_id → Razorpay SDK opens checkout → user pays → payment success/failure event fires → Razorpay sends webhook → backend verifies HMAC signature → updates donation status → frontend confirms completion.

Sequence Diagram: Details the technical interactions: Citizen's POST /donations/create-order request, backend's call to razorpay.Client.order.create(), return of order_id to client, Razorpay's asynchronous POST /donations/webhook to backend, and the idempotent database update.

4.3.2 Module 5: Smart NGO Dispatch & Hotspot Map

Use Case Diagram: Shows 'Admin' as the primary actor for 'View Hotspot Map', 'Select Case for Dispatch', and 'Assign NGO to Case'. The diagram also shows how the K-Means algorithm and Haversine formula feed into these use cases as backend services.

Activity Diagram: Two parallel workflows: (1) Admin views Zone Red Hotspot Map — backend fetches all case coordinates, runs K-Means clustering, returns cluster centroids with risk levels, frontend renders Leaflet.js map. (2) Smart Dispatch — Admin selects a case, backend ranks NGOs using Haversine + weighted scoring, frontend displays ranked table, Admin clicks 'Assign'.

Sequence Diagram: Detailed interaction flow: Admin's GET /admin/hotspots → backend queries cases with lat/lon → calls cluster_case_hotspots() → returns JSON centroids → frontend renders Leaflet.js circles. Simultaneously: Admin's GET /admin/smart-dispatch/{case_id} → backend queries case coords + all NGOs + active caseload counts → calls rank_ngos_for_case() → returns ranked list.

4.3.3 Module 6: AI Chatbot & LLM Integration

Sequence Diagram: Citizen types query → frontend sends POST /chatbot/query with query + full conversation history → backend calls groq.Client.chat.completions.create() with system prompt + history → Groq returns LLM response → backend returns response to frontend → frontend appends to conversation history → displayed in chat UI. On Groq API failure: backend catches exception and routes query to local chatbot_logic.py fallback, returning rule-based response.

4.3.4 Updated Class Diagram

The Phase II Class Diagram extends the Phase I model with two new entity classes: **Donation** (id, amount, currency, status, order_id, razorpay_payment_id, ngo_id FK, user_id FK, created_at) with many-to-one relationships to both User and NGO. **UserPetListing** (id, name, species, age, location, description, image_url, status, user_id FK, created_at) with a many-to-one relationship to User. The **Feedback** class gains two new attributes: category and ngo_response. The **Pet** class gains video_url and ngo_id FK.

4.3.5 Updated Component Diagram

The Phase II Component Diagram adds three new external service components: **Razorpay Payment Gateway** (interfaced via razorpay Python SDK and Razorpay Checkout.js client SDK), **Groq AI API** (interfaced via groq Python SDK), and **Google Auth Server** (interfaced via google-auth Python library and Google Sign-In JavaScript). The React Frontend gains new component groups: Charts (Recharts), Maps (Leaflet.js), and Analytics Dashboards.

Chapter 5: Rule-Based NLP vs. LLM-Based Approaches in Animal Emergency Platforms

5.1 Introduction

Natural Language Processing (NLP) enables systems to understand and interpret human language. In emergency response platforms like StrayCare, two complementary NLP paradigms are employed: (1) rule-based keyword matching for real-time, latency-critical case severity triaging, and (2) large language model (LLM) inference for nuanced, multi-turn first-aid conversational support. StrayCare Phase II implements both, assigning each to the task it is best suited for.

5.2 Rule-Based NLP for Emergency Severity Triaging

Historically, automated dispatch systems utilized simple keyword matching. Research on urban emergency response confirms that rule-based NLP provides the lowest latency — a critical metric when handling trauma cases. StrayCare's custom Rule-Based NLP engine in algorithms.py tokenizes the citizen's case description and applies a weighted scoring system:

- **Critical (+100 pts):** Keywords such as 'bleeding', 'unconscious', 'hit by car', 'paralyzed', 'seizure', 'not breathing', 'maggots', 'mauled', 'fractured', 'road accident'.
- **High (+60 pts):** Keywords such as 'injured', 'poisoned', 'trapped', 'bitten', 'attacked', 'infection', 'vomiting blood', 'convulsing'.
- **Moderate (+30 pts):** Keywords such as 'limping', 'malnourished', 'abandoned', 'mange', 'lethargic', 'scared'.

Score thresholds determine the severity label (Critical ≥100, High ≥60, Moderate ≥30, Low <30). Color-coded priority badges (red, orange, yellow, green) are then displayed on the NGO dashboard. This deterministic approach delivers instant classification without external API calls, ensuring sub-millisecond triaging even under high load.

5.3 LLM-Based Conversational Support (Phase II)

While rule-based systems excel at deterministic, real-time triaging, they fail to handle the nuanced, open-ended queries that concerned citizens and NGO workers ask in an emergency. "What should I do if my dog is in shock?" or "Can I give paracetamol to an injured cat?" require contextual reasoning that keyword matching cannot provide.

Phase II replaces the rule-based chatbot responses with the Groq API (Llama 3.3 70B model). Groq's hardware acceleration ensures sub-2-second response times for typical queries — faster than most hosted LLM APIs. The complete conversation history is passed to the model on every turn, enabling context-aware multi-turn dialogue. The system prompt configures the model as a specialized veterinary first-aid assistant, directing it to provide practical, safe, and actionable guidance while recommending professional veterinary care for serious injuries.

5.4 Hybrid Architecture: The Best of Both Worlds

Criterion	Rule-Based NLP (algorithms.py)	Groq LLM (Llama 3.3 70B)
Latency	< 1 ms	1–3 seconds
Use Case	Case severity triage at report submission	Multi-turn first-aid conversations
API Dependency	None — pure Python	Groq API (fallback: local)
Context Awareness	None — keyword only	Full conversation history
Cost	Zero	Groq API tokens (very low)
Determinism	Fully deterministic	Probabilistic (generative)
Maintainability	Manual keyword updates	System prompt tuning

Table 5.1: Comparison of Rule-Based NLP vs. Groq LLM in StrayCare Phase II

StrayCare's hybrid approach uses rule-based NLP where determinism and latency are paramount (automatic severity badge at case submission) and LLM where expressiveness and context-awareness matter (chatbot conversations). The LLM chatbot also includes a graceful fallback to the local rule-based engine in the event of Groq API unavailability, ensuring zero downtime for users.

Chapter 6: Machine Learning Concepts and Implementation

6.1 Overview of ML Mechanisms in StrayCare

StrayCare employs four distinct machine learning and mathematical mechanisms, all implemented natively without reliance on large external ML frameworks (except TensorFlow.js which runs in-browser):

- **Feature 1:** Haversine Distance Formula — Smart NGO Dispatch (geospatial algorithm)
- **Feature 2:** Pet Lifestyle Matchmaker — Compatibility scoring algorithm
- **Feature 3:** Rule-Based NLP Severity Triaging — Weighted keyword scoring with emergency classification
- **Feature 4:** K-Means Clustering — Zone Red Hotspot Map (unsupervised spatial ML)
- **Feature 5 (Phase II):** MobileNet V2 via TensorFlow.js — Animal species recognition (pre-trained CNN in-browser)
- **Feature 6 (Phase II):** Groq Llama 3.3 70B — Conversational AI for first-aid guidance (large language model, cloud inference)

6.2 Feature 1: Haversine Formula — Smart NGO Dispatch

The Haversine formula calculates the great-circle distance between two points on Earth's surface given their latitude and longitude coordinates. It is the geographically accurate alternative to the Euclidean distance formula, which fails on a spherical surface. In StrayCare, this algorithm is used to compute the distance between a reported emergency case and each registered NGO in the system:

Formula: d = 2R · arctan2(√a, √(1–a)), where a = sin²(Δϕ/2) + cos ϕ■ · cos ϕ■ · sin²(Δλ/2), R = 6371 km (Earth radius).

The dispatch scoring matrix then ranks NGOs by combining: Distance Score (0–50 pts, linearly decreasing over 200 km), Caseload Score (0–30 pts, linearly decreasing with active cases, max at 10), and Base Score (20 pts for all verified NGOs). The ranked list helps administrators dispatch emergencies to the most available, closest NGO in a single click.

6.3 Feature 2: Pet Lifestyle Matchmaker

This algorithm calculates a compatibility match percentage (15–99%) between an adoptable pet's profile and a prospective adopter's lifestyle. Starting from a base score of 60, adjustments are made based on:

- **Living Space:** Cats gain +15 pts in apartments; large dogs lose –30 pts in apartments; large dogs gain +15 pts in houses.
- **Activity Level:** Senior pets gain +20 pts for low-activity adopters; young/puppy/kitten pets lose –20 pts for low-activity adopters; adult dogs gain +15 pts for high-activity adopters.
- **Kids:** Large dogs near kids lose –10 pts (safety); medium/large dogs gain +5 pts (playmates for children).

- **Organic Variance:** A small deterministic perturbation based on pet ID prevents unrealistic ties and makes percentages feel natural.

The adoption gallery renders the match percentage as a colored badge (green = high match, yellow = moderate, red = low) for each pet, enabling data-driven adoption decisions.

6.4 Feature 3: Rule-Based NLP Severity Triaging

Covered in detail in Chapter 5. In summary: weighted keyword scoring produces a severity score (0 to potentially 500+), which maps to Critical/High/Moderate/Low labels and corresponding color badges. This runs entirely in Python server-side with no external dependencies, ensuring sub-millisecond performance even at high query rates.

6.5 Feature 4: K-Means Clustering — Zone Red Hotspot Map

K-Means Clustering is an unsupervised machine learning algorithm that partitions a set of data points into K clusters based on spatial proximity. StrayCare uses it to group GPS coordinates of all reported cases into geographic hotspot zones, identifying areas with concentrated animal rescue emergencies.

The implementation in algorithms.py runs the full K-Means iteration loop in pure Python: (1) Initialize K centroids by randomly sampling K points from the dataset (fixed seed=42 for reproducibility). (2) Assign each case point to its nearest centroid using Euclidean distance on lat/lon coordinates. (3) Recompute each centroid as the mean of all points assigned to it. (4) Repeat steps 2–3 until assignments do not change (convergence) or max_iterations (20) is reached. (5) Map each cluster's case count to a risk level: ≥10 cases = Critical Zone (red, 800m radius), ≥5 = High Zone (orange, 500m), ≥2 = Moderate Zone (yellow, 300m), 1 = Low Zone (green, 150m).

The Leaflet.js frontend renders cluster centroids as semi-transparent colored circles on an OpenStreetMap base layer, providing an immediate geographic overview of where emergency resources are most needed.

6.6 Feature 5: MobileNet V2 — In-Browser Species Recognition

A Convolutional Neural Network (CNN) deployed entirely in the citizen's browser via TensorFlow.js. When a citizen uploads a photo to the case report form, the AnimalRecognitionBadge component loads the pre-trained MobileNet V2 model (trained on ImageNet, 1000 categories). It classifies the photo and extracts animal-relevant predictions (dog, cat, bird, etc.), displaying a confidence badge (e.g., "Dog detected — 82% confidence") and auto-filling the case description. Key advantages: zero server roundtrips (privacy-preserving), instant inference, and works offline. MobileNet V2 achieves approximately 71.8% top-1 accuracy on ImageNet.

6.7 Performance Evaluation Metrics

Algorithm	Evaluation Metric	Target / Result

K-Means Clustering	Silhouette Score, Cluster Stability	Stable clusters on 10–500 case GPS coordinates; tested on synthe
Haversine Dispatch	Distance Accuracy (km)	< 0.1 km error verified against known Pune city coordinates
NLP Severity Triage	Label Accuracy	> 95% correct label on 50 manually pre-classified test descriptions
Pet Matchmaker	Score Range Validity	All scores between 15–99%, no out-of-range values on 200 rando
MobileNet V2	Top-1 Accuracy (ImageNet)	~71.8% (published benchmark), 82%+ on indoor animal photos
Groq Llama 3.3 70B	Response Latency, Coherence	< 2s p95 latency; domain-appropriate first-aid answers verified by

Table 6.1: Machine Learning Algorithm Performance Metrics

6.8 Enhancements and Future Work

• Transition rule-based NLP triaging to a fine-tuned DistilBERT or BERT-tiny model trained on verified animal rescue case descriptions for higher semantic accuracy.

• Integrate ARIMA or Prophet time-series forecasting with K-Means clusters to predict future accident hotspots based on historical seasonal trends (e.g., monsoon impact on stray animal injury rates).

• Replace MobileNet V2 with a custom-trained species classifier fine-tuned on Indian street animal images for higher domain-specific accuracy.

• Explore federated learning for training the severity classifier on NGO-submitted case data without centralizing sensitive incident descriptions.

Chapter 7: Software Testing and Quality Assurance

7.1 Introduction to Testing

To ensure the StrayCare platform is robust, secure, and capable of handling critical emergency data reliably, a comprehensive multi-layer testing strategy was implemented for Phase II. This strategy builds on the Phase I testing framework (unit testing, API endpoint testing, RBAC testing, E2E rescue lifecycle testing, cross-browser testing) and extends it with new test suites for the Razorpay payment gateway, Groq AI chatbot, analytics endpoints, and K-Means/Haversine algorithms.

7.2 Automated Testing — Phase II Additions

7.2.1 Algorithm Unit Testing (Phase II)

The following new functions in algorithms.py were unit tested:

- haversine_distance(lat1, lon1, lat2, lon2) — Tested with known city-pair coordinates: Mumbai-Pune (≈148 km), Pune-Delhi (≈1,413 km), verified within 0.1 km tolerance.
- rank_ngos_for_case(case_lat, case_lon, ngos, active_case_counts) — Verified that NGOs closer to the case always score higher in isolation; verified that high caseload correctly reduces score; verified consistent ranking across 10 random case locations.
- cluster_case_hotspots(cases, k) — Tested with input of 1, 5, 10, and 50 synthetic GPS points; verified convergence in ≤20 iterations; verified k is automatically capped to len(points); verified risk level assignment by case count.
- calculate_pet_match(pet, profile) — Verified score bounds [15, 99] across 50 random profile combinations; verified apartment+large_dog penalty; verified senior_pet+low_activity bonus.

7.2.2 API Endpoint Testing (Phase II)

Endpoint	Method	Expected Status	Test Scenario
/auth/google	POST	200 OK + JWT	Valid Google ID token
/auth/google	POST	401 Unauthorized	Tampered / expired token
/donations/create-order	POST	200 OK + order_id	Valid amount, valid NGO ID
/donations/create-order	POST	422	Missing amount field
/donations/webhook	POST	200 OK	Valid Razorpay webhook + correct HMAC
/donations/webhook	POST	400 Bad Request	Invalid HMAC signature
/admin/hotspots	GET	200 OK + clusters	Admin token + ≥1 case in DB
/admin/smart-dispatch/{id}	GET	200 OK + ranking	Valid case ID, NGOs in DB
/ngo/dashboard/cases/monthwise	GET	200 OK	NGO token, verified NGO
/admin/dashboard/cases	GET	200 OK	Admin token
/donor/verify/email/request	POST	200 + code	Authenticated citizen
/users/pets/list	POST	201 Created	Valid pet listing with image

/ngo/pets/listings	GET	200 OK	NGO token, pending listings exist

Table 7.1: Phase II API Endpoint Test Cases and Expected Outcomes

7.2.3 Payment Gateway Sandbox Testing

Manual testing in Razorpay's sandbox environment simulated the following scenarios:

- **Successful UPI Transaction:** Donation record created with status 'Created'; webhook delivered; status updated to 'Completed'; verified no duplicate record created on second webhook with same order_id.
- **Failed Card Transaction:** Webhook with event 'payment.failed' received; donation status updated to 'Failed' without corrupting existing records; frontend displayed appropriate failure message.
- **UPI Fallback Mode:** With RAZORPAY_KEY_ID unset, Donate button switched to deep-link UPI URL; verified link format: upi://pay?pa={upi_id}&am;={amount}&tn;=Donation.
- **Duplicate Webhook Idempotency:** Sent the same 'payment.captured' webhook twice for the same payment_id; verified database status updated only once (idempotency check via order_id lookup).

7.3 Manual Testing — Phase II

7.3.1 Groq AI Chatbot Testing

End-to-end chatbot testing covered:

- Multi-turn conversation: Verified conversation history is correctly passed to Groq API on each subsequent message, confirmed by contextually appropriate responses that referenced earlier turns.
- Boundary queries: Tested off-topic queries (weather, politics) — verified the chatbot politely redirected to animal rescue topics per the system prompt.
- Fallback behavior: Temporarily removed GROQ_API_KEY, sent a query — verified the backend caught the exception and returned a rule-based response with no 500 error exposed to the user.
- Response quality: Team veterinary consultation confirmed that first-aid advice for shock, bleeding, fractures, and poisoning cases was clinically appropriate.

7.3.2 Analytics Dashboard Testing

All three analytics dashboards were manually verified:

- NGO Analytics: Verified chart data updates correctly after accepting a new case, completing an adoption, and receiving a donation — all chart endpoints tested with 0, 1, and multiple records.
- Admin Analytics: Verified platform-wide totals reflect newly registered NGOs, new cases, and new donations in real time.
- Donor Dashboard: Verified public access (no authentication token required); confirmed month-wise donation chart correctly groups amounts by month; confirmed case stats reflect all cases regardless of status.

7.3.3 Zone Red Hotspot Map Testing

Tested with 0, 1, 5, and 20 case records with varying GPS coordinates. Verified: (1) empty state displays appropriate message with no map errors; (2) single point creates one 'Low Zone' cluster; (3) multiple points cluster correctly with risk levels matched to case counts; (4) map renders correctly on Chrome, Edge, Firefox, and Mobile Safari at 320px, 768px, and 1280px viewports.

7.3.4 End-to-End Phase II Workflow Testing

Evaluators tested the complete Phase II workflow:

- Citizen signs in via Google OAuth → directed to citizen dashboard → reports a case with animal photo → TensorFlow.js returns species badge → NLP assigns severity badge.
- Admin views Zone Red Hotspot Map → identifies high-risk cluster → opens Smart Dispatch for case → reviews NGO ranking list → assigns case to highest-ranked NGO.
- Citizen visits donation page → selects NGO → enters amount → Razorpay checkout opens → completes UPI payment → donation status shown as 'Completed'.
- Citizen views Donor Transparency Dashboard → verifies their donation appears in month-wise chart → sees total cases supported and adoptions facilitated.
- Citizen lists a pet for adoption → NGO reviews pending listing → approves listing → pet appears in adoption gallery.
- After case resolved, citizen leaves feedback → NGO views feedback → NGO submits a public response → citizen sees NGO response on case detail page.

Chapter 8: Applications of AI and ML in StrayCare

The combination of Rule-Based NLP, Haversine geospatial algorithms, K-Means clustering, TensorFlow.js in-browser CNN, and the Groq LLM within StrayCare demonstrates a multi-paradigm AI approach to civic technology:

- **1. Emergency Prioritization:** The Rule-Based NLP severity triager instantly classifies high-trauma cases (Critical, High) and elevates them to the top of every NGO's dashboard queue — without any manual review or external API calls. A description like 'dog hit by car, bleeding from head' is automatically scored as Critical (red badge) and given priority in the Smart Dispatch ranking.

- **2. Intelligent Resource Allocation:** The Haversine-based Smart Dispatch system prevents NGO burnout by routing emergencies to the organization with the best combination of proximity and available capacity. A 'Low' severity case far from all NGOs will not displace a nearby 'Critical' case in the dispatch ranking.

- **3. Proactive Civic Planning:** The K-Means Zone Red Hotspot Map enables municipalities and NGO networks to identify where to build new animal shelters, install road-crossing warning signs, or deploy mobile veterinary units based on statistically validated geographic cluster centroids — rather than anecdotal evidence.

- **4. Adoption Optimization:** The lifestyle-matching algorithm increases the probability of successful, lasting adoptions by quantifying the mathematical compatibility between a pet's characteristics and an adopter's living situation. Improving adoption outcomes directly reduces shelter overcrowding and animal return rates.

- **5. Citizen Empowerment through Conversational AI:** The Groq LLM chatbot provides evidence-based first-aid guidance to citizens in the critical window before professional help arrives. By handling open-ended, context-rich queries ("My cat is vomiting green fluid and shaking — what do I do?"), it goes far beyond what a rule-based system can achieve, potentially saving animal lives in the pre-rescue window.

- **6. Privacy-Preserving In-Browser Inference:** By running MobileNet V2 entirely in the citizen's browser via TensorFlow.js, animal photos are classified locally without being sent to any server for AI processing. This design choice protects user privacy while still delivering instant species detection and description auto-fill — a key user experience enhancement.

- **7. Data-Driven NGO Performance Monitoring:** The Admin Analytics Dashboard aggregates case acceptance rates, resolution times, adoption counts, and donation totals for every NGO. This enables evidence-based decisions about which organizations to prioritize for resource grants, partnership programs, or verification renewals.

Chapter 9: Methodology and Results

9.1 Technology Stack Details

The StrayCare Phase II technology stack is an evolution of the Phase I stack, with new integrations added at each layer:

Frontend Technology Stack:

- **React.js (v18.x):** Component-based SPA with useState, useEffect, and useRef hooks for state management.
- **CSS Modules:** Locally-scoped styles for every component, preventing global conflicts. Extended with new module files for DonatePage, NGOAnalyticsDashboard, CitizenDashboard, and AdminAnalyticsDashboard.
- **React Router DOM:** Client-side routing extended with new Phase II routes: /donate, /donor-dashboard, /hotspot-map, /smart-dispatch, /admin/analytics, /ngo/analytics, /donor/verify, /list-pet.
- **Axios:** All API calls use Axios with Authorization headers for JWT-protected endpoints.
- **Recharts:** SVG-based React charting library for all analytics dashboards. Used components: LineChart, BarChart, PieChart, AreaChart, ResponsiveContainer, Tooltip, Legend.
- **TensorFlow.js + MobileNet:** In-browser CNN for animal species recognition (Phase I, continued).
- **Leaflet.js + React-Leaflet:** Interactive map component for the Zone Red Hotspot Map. Uses OpenStreetMap as the base tile provider.
- **Razorpay Checkout.js:** Client-side Razorpay SDK loaded dynamically to open the payment checkout modal and handle payment events.

Backend Technology Stack:

- **FastAPI (Python):** RESTful API framework. Phase II added 25+ new endpoints across donation, analytics, dispatch, chatbot, OAuth, and user pet listing routers.
- **Uvicorn (ASGI):** Production-grade async server for FastAPI.
- **SQLAlchemy ORM + SQLite:** Unchanged from Phase I. New Phase II tables (donations, user_pet_listings) and column migrations applied at startup.
- **python-jose + passlib + bcrypt:** JWT-based authentication and password hashing (Phase I, continued).
- **Razorpay Python SDK:** Used for server-side order creation and HMAC-SHA256 webhook signature verification.
- **google-auth:** Verifies Google OAuth2 ID tokens server-side against the GOOGLE_CLIENT_ID.
- **Groq Python SDK:** Sends conversation history to the Llama 3.3 70B model via Groq's hosted inference API.
- **python-dotenv:** Loads all API keys and secrets from .env file at application startup.

9.2 Development Methodology — Phase II

Phase II follows the same Agile, iterative approach as Phase I, with work organized into two-week sprints for each major feature set:

- **Sprint 1 — Razorpay Donation Gateway:** Backend order creation, webhook handling, idempotent status tracking; Frontend DonatePage with NGO listing, Razorpay checkout integration, UPI fallback.
- **Sprint 2 — Google OAuth 2.0 & Groq AI Chatbot:** Backend /auth/google endpoint with google-auth token verification; Updated chatbot backend with Groq API call and fallback; Frontend Google Sign-In button in Login page.
- **Sprint 3 — Analytics Dashboards:** All NGO analytics endpoints (monthwise/yearwise cases, species, adoptions, donations); All Admin analytics endpoints; Donor public dashboard APIs; Frontend dashboards with Recharts.
- **Sprint 4 — Geospatial Intelligence:** K-Means clustering algorithm in algorithms.py; Haversine dispatch ranking algorithm; Backend /admin/hotspots and /admin/smart-dispatch/{case_id} endpoints; Frontend HotspotMap.jsx with Leaflet.js; Frontend SmartDispatch.jsx with ranking table.
- **Sprint 5 — User Pet Listings & NGO Feedback Response:** Backend user_pet_listing model, CRUD, and endpoints; NGO listing approval/rejection endpoint; Backend /feedback/{id}/respond endpoint; Frontend UserPetListingForm, NGOPetListings, updated NGOFeedback with reply modal.
- **Sprint 6 — Integration, Testing & Documentation:** End-to-end integration testing across all Phase II features; Sandbox payment testing; Cross-browser and responsive UI testing; Project report writing.

9.3 Results — Phase II Module Screenshots

9.3.1 Donation Module

Figure 9.1 — Donate Page (NGO Selection)

Figure 9.1 shows the Donate Page, which lists all verified NGOs with their 30-day donation total and key statistics (cases handled, adoptions facilitated). Citizens select an NGO and enter a donation amount, after which the Razorpay Checkout SDK opens for secure payment processing. A UPI deep-link button is also displayed for users preferring direct UPI transfers.

Figure 9.2 — Donor Transparency Dashboard

Figure 9.2 shows the public Donor Transparency Dashboard, accessible without authentication. It displays a month-wise donation bar chart, total donation amount raised, total cases supported (all statuses), and total pet adoptions facilitated. This dashboard builds public trust and encourages repeat donations by demonstrating measurable animal welfare impact.

Figure 9.3 — Donor KYC Verification Page

Figure 9.3 shows the Donor Verification Page, which guides authenticated citizens through a two-step OTP-based verification process: email verification (OTP generated server-side) and phone number verification. Upon successful completion, the user's donor_verified status is updated, enabling higher donation thresholds and priority donor status.

9.3.2 Analytics Dashboards

Figure 9.4 — NGO Analytics Dashboard

Figure 9.4 shows the NGO Analytics Dashboard, which provides the NGO with a comprehensive visual overview of their operations. Four key charts are displayed: a line chart of monthly case trends (current year), a bar chart of accepted cases by year, a pie chart of cases by animal species detected via TensorFlow.js, and a bar chart of monthly donations received. Each chart uses a consistent color scheme aligned with the StrayCare brand palette.

Figure 9.5 — Admin Analytics Dashboard

Figure 9.5 shows the Admin Analytics Dashboard, giving administrators a platform-wide operational view. It displays total cases by status (Pending, Accepted, Rejected, Resolved) in a bar chart, total donations amount over time in an area chart, and platform-wide adoption counts in a pie chart. Summary KPI cards at the top display key totals at a glance.

9.3.3 Geospatial Intelligence

Figure 9.6 — Zone Red Hotspot Map

Figure 9.6 shows the interactive Zone Red Hotspot Map, rendered using Leaflet.js on an OpenStreetMap base layer. Color-coded semi-transparent circles represent K-Means cluster centroids: red circles (Critical Zones, ≥10 cases, 800m radius) appear in areas with the highest emergency density; orange circles (High Zones, ≥5 cases, 500m radius), yellow (Moderate, ≥2, 300m radius), and green (Low, 1 case, 150m radius). Clicking a circle displays the cluster's case count and risk level in a popup.

Figure 9.7 — Smart NGO Dispatch Interface

Figure 9.7 shows the Smart NGO Dispatch Interface. On the left, the selected case details (GPS coordinates, description, severity badge) are displayed. On the right, the ranked NGO recommendation table shows each NGO's name, computed distance (km), active case count, proximity score, caseload score, and total dispatch score. The highest-ranked NGO is highlighted and a single "Assign" button dispatches the case to that organization.

9.3.4 AI Chatbot

Figure 9.8 — Groq AI-Powered Chatbot Interface

Figure 9.8 shows the upgraded AI Chatbot interface. The chat panel displays a conversation history with alternating citizen and AI response bubbles. The chatbot leverages Groq Llama 3.3 70B to provide detailed, multi-turn first-aid guidance — for example, advising a citizen on how to safely transport an injured dog with a suspected spinal injury, referencing the previous turns in the conversation for context. The AI badge in the header indicates the active model and includes a fallback indicator when operating in offline mode.

9.3.5 Citizen Pet Listing

Figure 9.9 — Citizen Pet Listing Form

Figure 9.9 shows the Citizen Pet Listing Form, allowing citizens to submit a pet for community re-homing. Fields include pet name, species, age, location, and a description, along with a photo upload. Upon submission, the listing enters a 'Pending NGO Review' state. Once an NGO approves the listing, the pet appears in the public adoption gallery alongside NGO-listed pets.

Chapter 10: Conclusion

StrayCare Phase II represents a transformative evolution from a working prototype into a comprehensive, production-ready civic technology platform for animal welfare. By systematically addressing the gaps identified after Phase I — lack of financial infrastructure, limited AI intelligence, absence of analytics, no geospatial tooling — Phase II has delivered a fully integrated ecosystem that serves citizens, NGOs, administrators, and donors with purpose-built, role-specific tools.

The integration of the Razorpay payment gateway establishes a trustworthy, verifiable donation channel that creates a direct financial lifeline between compassionate donors and frontline rescue organizations. The Groq LLM chatbot transforms the platform's conversational capability from simple keyword responses into nuanced, contextually aware veterinary guidance that can meaningfully support citizens in the critical pre-rescue window. Google OAuth 2.0 eliminates authentication friction, making it easier for more citizens to engage with the platform.

The three analytics dashboards — NGO, Admin, and Donor — transition StrayCare from an operational tool into a data intelligence platform. By visualizing case trends, geographic hotspots, donor impact, and NGO performance, these dashboards enable evidence-based resource allocation and operational planning that was previously impossible. The Zone Red Hotspot Map and Smart NGO Dispatch system embody the platform's mission to apply data science directly to the urgent problem of animal rescue logistics.

The citizen pet listing feature broadens the adoption ecosystem beyond NGO-managed animals, enabling community members to directly participate in responsible pet rehoming under NGO oversight. NGO feedback responses complete the accountability loop that Phase I began, creating a transparent review ecosystem that rewards high-performing organizations.

From an engineering perspective, Phase II demonstrates best practices in API-first development, safe database migration, graceful degradation (LLM fallback, UPI deep-link fallback), idempotent webhook processing, and privacy-preserving in-browser AI inference. The codebase remains modular, maintainable, and extensible — ready for the future enhancements outlined in the ML chapter, including transformer-based NLP triaging and predictive hotspot forecasting.

Ultimately, StrayCare Phase II is a testament to how thoughtful full-stack software engineering, combined with modern AI and financial APIs, can address real-world civic challenges at scale — transforming community empathy into measurable, verifiable animal welfare impact.

Chapter 11: References

1. R. Kolandaisamy, K. Subramaniam, I. Kolandaisamy, and L. S. Li, "Stray Animal Mobile App," UCSI University, Kuala Lumpur, Malaysia, 2016.
2. A. Abdulkarim, M. A. K. B. G. Khan, and E. Aklilu, "Stray Animal Population Control: Methods, Public Health Concern, Ethics, and Animal Welfare Issues," World Veterinary Journal, vol. 11, no. 3, pp. 319–326, 2021.
3. L. L. A. Walter, "StandForPaw: Animal Rescue and Pet Adoption Mobile Application," Universiti Teknologi PETRONAS, Malaysia, 2021.
4. C. L. Yi and C. S. C. Dalim, "Development of Mobile Application for Animal Conservation: Animal Rescue," Applied Information Technology and Computer Science, vol. 4, no. 2, pp. 430–449, 2023.
5. Y. H. Jean and N. Wahid, "FurRescue: A Mobile Application for Pet and Stray Animal Locator with Geo-Fencing and AI Breed Detection," Applied Information Technology and Computer Science, vol. 6, no. 1, pp. 1–20, 2025.
6. Wu et al., 2022 — "Widespread of Stray Animals: Design a Technological Solution to Help Build a Rescue System for Stray Animals."
7. Neha Jha et al., 2024 — "Using Machine Learning and AI to Find Homes for the Voiceless."
8. Sengan et al., 2021 — "Real-Time Automatic Investigation of Indian Roadway Animals by 3D Reconstruction Detection Using Deep Learning for R-3D-YOLOv3."
9. Rahajeng et al., 2024 — "Mobile Application to Help Abandoned Pets."
10. Blancaflor et al., 2023 — "CareForPaws: A Mobile Application for Pet Adoption and Other Services with Location-Based Technology."
11. Bricman & Kozuh, 2024 — "User Experience and Interface Design for a Digital Pet Adoption Platform (PetScout)."
12. Jay Prajapati et al., 2025 — "Pet Adoption and Rescue Center."
13. Mahak Rajwani et al., 2025 — "Enhancing Animal Rescue Efficiency Using AI for Real-Time Reporting and Assistance."
14. Honglin Lu, 2025 — "Pet Adoption Status Prediction Based on Multiple Machine Learning Models."
15. Kalaivani A/P Subramaniam & Hazinah Kutty Mammi, 2025 — "Stray Rescue Management System (SRMS)."
16. Torres et al., 2024 — "A Web-Based Animal Adoption and Rescue System that uses Content-Based Filtering Algorithm."
17. Jukan, Masip-Bruin & Amla, 2016 — "Smart Computing and Sensing Technologies for Animal Welfare: A Systematic Review."
18. FastAPI Documentation: https://fastapi.tiangolo.com/
19. TensorFlow.js Models & MobileNet: https://www.tensorflow.org/js
20. Razorpay API Documentation: https://razorpay.com/docs/
21. Groq API Documentation & Llama 3.3 70B: https://console.groq.com/docs/
22. Google Identity Services (OAuth 2.0): https://developers.google.com/identity

23. Leaflet.js Interactive Map Library: https://leafletjs.com/
24. Recharts — Redefined chart library built with React and D3: https://recharts.org/
25. OpenStreetMap Tile Server: https://www.openstreetmap.org/

Chapter 12: Project Plan — Phase II

Week	Activity Planned / Module(s) Covered	Status
Week 1	Phase II Requirements Analysis — Identify Phase I gaps; define Phase II scope: Razorpay donations, Google	Completed
Week 2	Database Schema Design (Phase II) — Design Donation and UserPet models; plan column migrations f	Completed
Week 3	Razorpay Backend Integration — Implement POST /donations/create-order, POST /donations/webhook; integr	Completed
Week 4	Razorpay Frontend Integration — Build DonatePage.jsx with NGO listing, amount input, Razorpay Checkout S	Completed
Week 5	Google OAuth 2.0 & Groq AI Chatbot — Backend: /auth/google endpoint; frontend: Google Sign-In button. Bac	Completed
Week 6	Donor Verification & Public Donor Dashboard — Implement OTP-based email/phone verification endpoints. Bu	Completed
Week 7	NGO Analytics Dashboard — Implement all /ngo/dashboard/* endpoints (monthwise cases, yearwise cases, sp	Completed
Week 8	Admin Analytics Dashboard — Implement all /admin/dashboard/* endpoints. Build AdminAnalyticsDashboard.js	Completed
Week 9	K-Means Hotspot Algorithm & Map — Implement cluster_case_hotspots() in algorithms.py. Add GET /admin/ho	Completed
Week 10	Smart NGO Dispatch Algorithm & UI — Implement haversine_distance() and rank_ngos_for_case() in algorithm	Completed
Week 11	Citizen Pet Listings & NGO Feedback Response — Implement UserPet model, CRUD, and endpoints. B	Completed
Week 12	Integration & Regression Testing — End-to-end testing of all Phase II workflows; Razorpay sandbox testing; G	Completed
Week 13	Final Documentation — Write complete Phase II project report; prepare Phase II presentation; record applicatio	Completed

Table 12.1: StrayCare Phase II Project Plan and Completion Status